SIMATS ENGINEERING

ASSESSMENT TOOL 2 – ANSWERS

Course Name: Artificial Intelligence **Course Code:** CSA17

Topic: Constraint-Based Problem Solving

Name: A.K.Kishore Kumar **Register No:** 1913260079

Code of Conduct: I, A.K.Kishore Kumar (1913260079), certify that this submission is my original work and that I have followed the guidelines specified for this assessment.

Signature: _____

Question 1: Backtracking Search for Doctor Shift Assignment

Constraints

1. D1 cannot work the Night shift.
2. D2 must work before D3 (D2's shift must be earlier than D3's).
3. Only one doctor can work each shift.
4. D3 cannot work the Morning shift.
5. Each doctor must be assigned exactly one shift.

Step-by-step backtracking and constraint checks

Step	Assignment / trial	Constraint check and result
1	D1 = Morning	Valid. D1 is not assigned to Night.
2	D2 = Afternoon	Valid so far. D2 is assigned before D3 if D3 is placed at Night.
3	D3 = Night	Valid. D3 is not in Morning, and D2 is before D3.
4	Check all shifts	Morning, Afternoon and Night each have one doctor.
5	Check all doctors	D1, D2 and D3 each have exactly one shift. All constraints are satisfied.

Rejected branches: D1 = Night is rejected because D1 cannot work Night. D3 = Morning is rejected because D3 cannot work Morning. A branch is abandoned whenever it violates a constraint; the search then backtracks and tries another assignment.

Final valid schedule: Morning – D1 | Afternoon – D2 | Night – D3

Question 2: Grid Navigation Using BFS

Note: The question states that the environment is a 5×5 grid but does not show the actual positions of S, G, or the obstacles. Therefore, the exact path and cost cannot be uniquely determined from the supplied assessment. The grid below is only an illustrative example to demonstrate the algorithm.

Illustrative grid (S = Start, G = Goal, # = obstacle, · = free cell)

S	·	·	#	·
#	·	#	·	·
·	·	·	·	#
·	#	#	·	·
·	·	·	·	G

Algorithm and heuristic

Breadth-First Search (BFS) uses a FIFO queue and expands nodes level by level. With every move costing 1, BFS finds a path with the minimum number of moves. The Manhattan distance is h(n) = |row(n) – row(G)| + |column(n) – column(G)|. It can be calculated for reference, but standard BFS does not use this heuristic to prioritize nodes; A* uses both path cost and a heuristic.

Step-by-step node expansion

For the example grid, use coordinates (row, column), with (1,1) as S. Check neighbours in the order Right, Down, Left, Up. Ignore cells that are blocked, outside the grid, or already visited. The queue shown is FIFO.

Step	Expanded node	Queue additions
1	(1,1) S	(1,2)
2	(1,2)	(1,3), (2,2)
3	(1,3)	No new node; right cell is blocked and other valid neighbours are already visited / blocked.
4	(2,2)	(3,2)
5	(3,2)	(3,3), (3,1)
6	(3,3)	(3,4)
7	(3,1)	(4,1)
8	(3,4)	(4,4), (2,4)
9	(4,1)	(5,1)
10	(4,4)	(4,5), (5,4)
11	(2,4)	(2,5)
12	(5,1)	(5,2)
13	(4,5)	(5,5) = G; goal reached

Priority queue clarification: BFS does not use a priority queue; it uses a FIFO queue. If the assessment specifically requires a priority queue ordered using Manhattan distance, that would be a heuristic-based search rather than standard BFS.

Example path and cost: One shortest path in the illustrative grid is (1,1) → (1,2) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4) → (4,5) → (5,5). Total cost = 8 moves × 1 = 8. This result applies only to the example grid above, not necessarily to an omitted original grid.

Question 3: Autonomous Rescue Robot in a Partially Observable Building

(a) Analysis of constraints

1. **Four-direction movement:** The robot may move only up, down, left, or right, so diagonal shortcuts are not allowed.
2. **Obstacles:** Blocked cells must be excluded from candidate routes.
3. **Limited sensing:** The robot observes only adjacent cells and cannot know the full map at the beginning. It must use local observations to update its internal map.
4. **Movement cost:** Every move costs 1, while entering a risky zone adds 2, making that move cost 3 in total.
5. **Cost minimization and safety:** The robot should compare safe and risky routes and choose the lowest-cost feasible route, avoiding blocked cells and unnecessary risk.
6. **Online knowledge updates:** The robot must revise its map and decisions whenever it discovers a new obstacle or risk.

(b) Suitable approach: Online search with Uniform Cost Search (UCS)

UCS is suitable for finding the lowest-cost route when actions have different costs. An online search agent is needed because the robot cannot observe the whole environment in advance.

Steps

1. Initialize the robot's known map with its starting position S; mark unknown cells as unknown.
2. Sense the adjacent cells and identify which are free, blocked, or risky.
3. Add reachable known cells to a frontier ordered by accumulated path cost g(n).
4. Select the frontier node with the lowest g(n), then consider its legal neighbours.
5. Add cost 1 for a normal move and cost 3 for entering a risky cell (base cost 1 + additional cost 2). Never enter a blocked cell.
6. When new information is sensed, update the map and recalculate affected routes. If a planned route is blocked, replan from the current position.
7. Stop when the robot reaches G. The resulting route is optimal with respect to the discovered map and costs; if unseen areas remain, continued sensing and replanning may be necessary.

(c) Application of AI concepts

• **Intelligent agent:** The robot senses nearby cells, maintains an internal representation of the environment, chooses an action, and moves toward the survivor.
• **Rationality:** A rational robot selects the action expected to minimize total travel cost while respecting safety constraints and using the information currently available. It cannot act on information it has not sensed.
• **Environment type:** The environment is partially observable, sequential, dynamic in the robot's knowledge, and includes

deterministic movement with weighted action costs. It is also discrete because positions are grid cells and actions are four-direction moves. The world may be unknown initially, so the robot uses online search.

(d) Justification

UCS is appropriate because the actions do not all have the same effective cost: entering a risky zone costs 3, while a normal move costs 1. Unlike BFS, UCS accounts for these different costs and expands the lowest accumulated-cost option first. Combining UCS with an online search agent allows the robot to learn the map through adjacent sensing and replan when new obstacles or risks are discovered. This supports cost-effective and safer navigation toward the survivor.